

# UVa1341/LA3262 Different Digits

## 题目链接

本题是 [2004 年 ICPC 亚洲区域赛 上海赛区](#) 的题目

## 题意

输入正整数  $n$  ( $n < 65536$ )，求它的最小倍数  $m$ ，使得  $m$  包含的不同的数字种类尽量少（例如，1334 有 3 种数字：1, 3, 4）。如果有多解， $m$  的值应尽量小。例如， $n=101$  时答案为 1111。

## 分析

非常好的一道数论综合题目，涉及数论分析、剩余系、欧拉降幂、枚举、最短路等知识点。

首先，如果  $n$  恰好包含一种数字，答案就是  $n$ 。

如果  $n$  为质数，可以发现答案恰好包含一种数字（ $n=101$  时答案为 1111， $n=13$  时答案为 111111）。

$n$  为合数时，要考虑其因子情况（含 2、含 5、其他），因子仅包含一个 2 不包含 5（可以包含其他质数）或者仅包含一个 5 不包含 2 时，答案仍然只包含一种数字（比如  $n=26$  时答案为 222222， $n=35$  时答案为 555555），其他时候的答案最多也只包含两种数字。

什么时候答案包含两种数字呢？其实也很明确： $n$  能被 10 或 16 或 25 整除。

总结一下： $n$  能被 10 或 16 或 25 整除答案恰好包含两种数字，否则为答案仅包含一种数字。

题目还要考虑多解时  $m$  的值尽量小，答案仅仅包含一种数字时的求解可以这样做：依次枚举位数  $x$ （再枚举数字  $d$ ），先求出  $m=11\dots1$ （连续  $x$  个 1）时对  $n$  的余数  $r$ （解借助欧拉降幂），如果  $r==0$  这就是答案了，不为零时  $++d$ ，检查  $d*r\%n$  是否为零更新答案。

答案仅包含两种数字时，首先如果  $n$  恰好包含 2 种数字，答案就是  $n$ ；否则，仍然借鉴前一种枚举方案求解：答案一定可以表示成  $dd\dots d$ （连续  $x$  个 1）+  $e??\dots?$ （ $?$  代表  $e$  或者 0， $e??\dots?$  的总位数不超过  $x$  并且  $d+e \leq 9$ ）。

求出了  $dd\dots d$  对  $n$  的余数  $r$ ，那么  $e??\dots?$  对  $n$  的余数一定为  $n-r$ 。

怎么找出余数为特定值时  $e??\dots?$  的最小值呢？用 dp 解决：设  $\text{least}[e][r]$  表示  $e??\dots?$  对  $n$  的余数为  $r$  时，最高位的  $e$  所在的最小位置，则根据余数关系即可做状态转移。

实际求解中为了优化效率还需要按  $e??\dots?$  的位数小于  $x$  和等于  $x$  两种情况处理：先计算  $e??\dots?$  的位数小于  $x$  时答案的最高位值  $d$ ，再枚举  $e??\dots?$  的位数等于  $x$  时的  $dd\dots d$  的实际位  $d_1$  ( $d_1 + e \leq d$ )。

更多细节，详见 AC 代码。

## AC 代码

## C/C++

```
1 #include <iostream>
2 #include <cstring>
3 #include <string>
4 using namespace std;
5
6 #define N 66000
7 int r[N<<1], least[10][N], n;
8
9 int digits(int x) {
10     bool f[10] = {0}; int cnt = 0;
11     while (x > 0) f[x%10] = true, x /= 10;
12     for (int i=0; i<10; ++i) if (f[i]) ++cnt;
13     return cnt;
14 }
15
16 int phi() {
17     int x = n, ans = n;
18     for (int i=2; i*i <= x; ++i) if (x%i == 0) {
19         ans = ans/i*(i-1);
20         while (x%i == 0) x /= i;
21     }
22     if (x > 1) ans = ans/x*(x-1);
23     return ans;
24 }
25
26 int pow(int x, int phi) {
27     return r[x <= phi ? x : x%phi + phi];
28 }
29
30 int find(int d, int r) {
31     int x = 0;
32     for (int i=1; i<d; ++i) if (least[i][r]) {
33         if (!x || least[i][r] < least[x][r]) x = i;
34     }
35     return x;
36 }
37
38 void sol() {
39     memset(least, 0, sizeof(least));
```

```

40     int d = 0, p = phi(), sr = 1; string ans;
41     r[0] = 1; for (int i=1, q=p<<1; i<q; ++i) r[i] = 10*r[i-1]%n;
42     for (int i=1; !d; sr = (sr + pow(i++, p)) % n) {
43         for (int j=1; !d && j<10; ++j) {
44             int v = n - sr*j % n, x = find(10-j, v);
45             if (x) {
46                 char ch = '0' + (d = j);
47                 for (int k=0; k<i; ++k) ans += ch;
48                 for (int y; y = least[x][v]; v = (v - pow(y-1, p)*x%n
+ n) % n) ans[i-y] += x;
49             }
50         }
51         for (int j=1, v; j<10; ++j) {
52             for (int k=0; k<n; ++k) least[0][k] = least[j][k];
53             if (!least[j][v = pow(i-1, p)*j%n]) least[j][v] = i;
54             for (int k=1, x; k<n; ++k) if (least[0][k] && !least[0][x=
(k+v)%n]) least[j][x] = i;
55         }
56         for (int j=1, x = d ? d+1 : 10; j<x; ++j) for (int k=0, v,
y; k<j; ++k)
57             if (least[y = j-k][(v = n-sr*k%n) % n] == i) {
58                 char ch = '0' + k;
59                 if (d) for (int q=0; q<i; ++q) ans[q] = ch;
60                 else for (int q=0; q<i; ++q) ans += ch;
61                 for (int z=i; v; z = least[y][v = (v - pow(z-1, p)*y%n +
n) % n]) ans[i-z] += y;
62                 x = d = j;
63                 break;
64             }
65     }
66     cout << ans;
67 }
68
69 void solve() {
70     int c = digits(n);
71     if (c > 1) {
72         if (n%10==0 || n%16==0 || n%25==0) {
73             if (c > 2) sol();
74             else cout << n;
75         } else {
76             int d = 0, r = 1, sr = 1;

```

```
77         for (int i=2; !d; ++i) {
78             r = 10*r%n; sr = (sr + r) % n;
79             for (int j=1; j<10; ++j) if (sr*j%n == 0) {
80                 d = j;
81                 for (int k=0; k<i; ++k) cout << d;
82                 break;
83             }
84         }
85     }
86     } else cout << n;
87     cout << endl;
88 }
89
90 int main() {
91     while (cin>>n && n) solve();
92     return 0;
93 }
```